

Rapport d'Architecture Technique

Résolution des instabilités WebSocket et Optimisation Temps Réel

Département Ingénierie Logicielle

Juillet 2026

Table des matières

1	Introduction	2
2	Partie 1 : Saturation de la boucle d'événements FastAPI (Event Loop Starvation)	2
2.1	Le Problème en Détail	2
2.2	La Solution Appliquée	2
2.2.1	Pourquoi le Multi-threading fonctionne-t-il ici ?	3
3	Partie 2 : Boucle de Reconnexion infinie sur le Frontend (Next.js)	3
3.1	Le Problème en Détail	3
3.2	La Solution Appliquée	4
4	Conclusion	5

1 Introduction

Dans le cadre du développement de notre pipeline d'ingestion de données agronomiques en temps réel, notre architecture repose sur une transmission d'événements asynchrones via Apache Kafka, orchestrée par un backend **FastAPI** qui diffuse ces métriques prédictives à un tableau de bord **Next.js** via un serveur **Socket.io** (WebSockets).

Ce rapport documente l'analyse post-mortem de deux instabilités majeures survenues lors de la phase d'intégration : l'asphyxie de la boucle d'*event-loop* FastAPI lors de l'inférence par les modèles de Machine Learning (XGBoost et MLP), et la reconnexion en boucle infinie des sockets côté client lors de la mise à jour de l'état UI.

2 Partie 1 : Saturation de la boucle d'événements FastAPI (Event Loop Starvation)

2.1 Le Problème en Détail

La spécification d'Asynchronous Server Gateway Interface (ASGI) utilisée par FastAPI repose sur une boucle d'événements mono-threadée (*asyncio event loop*). Cette boucle gère de manière concurrente toutes les opérations d'E/S (comme les connexions Socket.io, la réception des messages Kafka, etc.) en passant d'une tâche à l'autre dès qu'un mot-clé `await` (opération non bloquante) est rencontré.

Cependant, les prédictions de Machine Learning via XGBoost (`xgboost_model.predict`) et les perceptrons multicouches (`nn_mlp_model.predict`) sont des tâches **calculatoires lourdes (CPU-bound)**. Contrairement aux opérations d'E/S, ces calculs mathématiques s'exécutent entièrement sur le processeur sans jamais rendre le contrôle de manière coopérative à la boucle d'événements.

Mécanisme de Déconnexion Socket.io

Pour maintenir une connexion WebSocket active, Socket.io utilise un mécanisme de *Heartbeat* bidirectionnel (Ping/Pong).

1. Le client ou le serveur envoie un paquet **Ping** à intervalles réguliers (ex. toutes les 25 secondes).
2. Le récepteur doit répondre immédiatement par un paquet **Pong**.
3. Lorsque FastAPI traite l'inférence des modèles sur le thread principal, l'ensemble de l'*event loop* est **gelé**.
4. Durant ce gel, le serveur est incapable de répondre au Ping. Une fois le `ping_timeout` dépassé (ex. 20 secondes), le client déclare la connexion morte et initie une déconnexion forcée suivie d'une tentative de reconnexion.

2.2 La Solution Appliquée

Pour résoudre ce blocage, nous avons extrait l'exécution des fonctions CPU-bound du thread principal de la boucle d'événements asynchrones en les déléguant à un **Pool de Threads** en arrière-plan via `asyncio.to_thread`.

2.2.1 Pourquoi le Multi-threading fonctionne-t-il ici ?

Bien que Python soit soumis au *Global Interpreter Lock* (GIL) qui restreint l'exécution de bytecode Python à un seul thread à la fois, les bibliothèques d'analyse de données et de Machine Learning majeures (telles que NumPy, Scikit-Learn, PyTorch, C++ XGBoost) écrivent leurs opérations bas niveau en C/C++. Pendant l'exécution de ces calculs matriciels intensifs, ces bibliothèques **libèrent le GIL**, permettant à de multiples threads physiques de s'exécuter de façon véritablement parallèle.

De plus, nous avons optimisé le pipeline en parallélisant les deux prédictions indépendantes à l'aide d'`asyncio.gather`, réduisant ainsi le temps de latence total à $\max(T_{\text{xgb}}, T_{\text{mlp}})$ au lieu de la somme séquentielle $T_{\text{xgb}} + T_{\text{mlp}}$.

```

1  async def ml_analytics(data_dict: dict, sio, xgboost_model, nn_mlp_model
2      ):
3      try:
4          # Transformation Pandas rapide (non-bloquante)
5          raw_time_data = pd.DataFrame([data_dict])
6
7          # Exécution en parallèle dans des threads séparés
8          xgb_pred, nn_pred = await asyncio.gather(
9              asyncio.to_thread(xgboost_model.predict, raw_time_data),
10             asyncio.to_thread(nn_mlp_model.predict, raw_time_data)
11         )
12
13         xgb_val = xgb_pred.tolist() if hasattr(xgb_pred, "tolist") else
14         xgb_pred
15         nn_val = nn_pred.tolist() if hasattr(nn_pred, "tolist") else
16         nn_pred
17
18         payload = {
19             "weather": data_dict,
20             "time": datetime.now().isoformat(),
21             "xgboostPrediction": xgb_val[0] if isinstance(xgb_val, list)
22             else xgb_val,
23             "nnMlpPrediction": nn_val[0] if isinstance(nn_val, list)
24             else nn_val,
25         }
26
27         await sio.emit("weather-emit", {"data": payload})
28     except Exception as e:
29         print(f"Erreur d'inférence: {e}")

```

Listing 1 – Implémentation asynchrone parallélisée avec `asyncio.to_thread`

3 Partie 2 : Boucle de Reconnexion infinie sur le Frontend (Next.js)

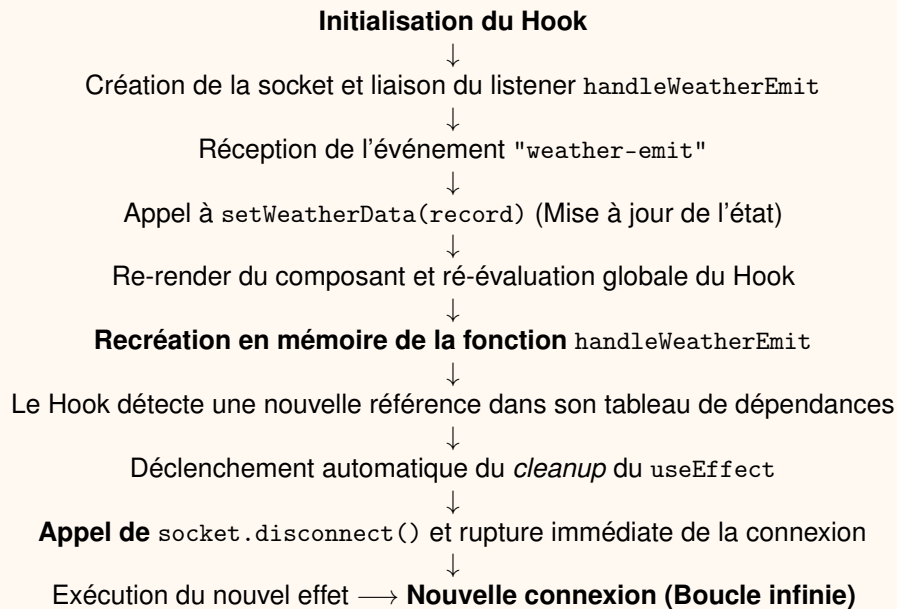
3.1 Le Problème en Détail

Une fois la boucle d'événements backend libérée, les logs montraient toujours une reconnexion immédiate du client dès la réception des données : un client se connectait, recevait les métriques, se déconnectait instantanément, puis se reconnectait avec un nouvel identifiant (Session ID).

La cause racine de cette anomalie résidait dans l'implémentation du hook React personnalisé `useSocketWeather` et la gestion de son cycle de vie.

Cycle d'instabilité généré par la dépendance fonctionnelle

Voici l'enchaînement d'événements qui provoquait cette déconnexion :



Le problème venait du fait que le gestionnaire d'événements (`handleWeatherEmit`) était déclaré directement dans le corps du hook et passé dans le tableau de dépendances du `useEffect`. À chaque re-render, la référence de la fonction changeait, forçant la destruction et la reconstruction immédiate de la socket.

3.2 La Solution Appliquée

Pour garantir que la socket ne se connecte qu'une seule fois à l'initialisation et survive aux cycles de rendu de l'interface utilisateur, nous avons appliqué les corrections suivantes :

- Encapsulation locale** : La fonction de callback `handleWeatherEmit` a été déplacée à l'intérieur du `useEffect`. Elle n'est ainsi plus déclarée dans le scope global du hook à chaque rendu, supprimant le besoin de l'ajouter aux dépendances.
- Isolation des dépendances** : Le tableau de dépendances du `useEffect` ne surveille plus que l'URL racine de l'API (`[url]`), qui reste parfaitement constante au cours du cycle de vie de l'application.
- Correction d'événement** : Correction de l'écouteur d'état Socket.io : l'événement natif de déconnexion de Socket.io-client est "disconnect" (et non "disconnected").

```

1 import { io } from "socket.io-client";
2 import { useEffect, useState } from "react";
3
4 export default function useSocketWeather({ url }: { url: string }) {
5   const [status, setStatus] = useState("disconnected");
6   const [weatherData, setWeatherData] = useState(null);
7

```

```
8      useEffect(() => {
9          if (!url) return;
10
11          // Connexion unique à l'URL
12          const socket = io(url, {
13              path: "/ws/socket.io/",
14              transports: ["websocket"],
15              withCredentials: true,
16          });
17
18          socket.on("connect", () => setStatus("connected"));
19          socket.on("disconnect", () => setStatus("disconnected"));
20
21          // Handler encapsulé à l'intérieur de l'effet
22          const handleWeatherEmit = (payload) => {
23              const { weather, xgboostPrediction, nnMlpPrediction, time }
24              = payload.data;
25              setWeatherData({
26                  weather,
27                  time,
28                  xgboostPrediction,
29                  nnMlpPrediction,
30              });
31          };
32          socket.on("weather-emit", handleWeatherEmit);
33
34          // Nettoyage propre au démontage du composant uniquement
35          return () => {
36              socket.off("weather-emit", handleWeatherEmit);
37              socket.disconnect();
38          };
39      }, [url]); // Seule la mutation d'URL peut déclencher une
40      reconnexion
41
42      return { weatherData, status };
43  }
```

Listing 2 – Version corrigée et stable du Hook useSocketWeather

4 Conclusion

En découplant les tâches bloquantes CPU-bound du thread principal de FastAPI à l'aide d'un pool de threads asynchrones d'une part, et en encapsulant proprement les écouteurs d'événements à l'intérieur du cycle d'*effet* de React d'autre part, nous avons éliminé les goulots d'étranglement de notre pipeline.

La connexion WebSocket est désormais parfaitement stable, persistante, et le traitement concurrent permet d'assurer un affichage fluide de nos métriques d'irrigation à une fréquence optimale sans aucune dégradation de performances.